CS34020/CSM4220 Workshop 2

Edges and Features

Theme: Bioinformatics

Purpose and theme: in this workshop we will explore different forms of edge and feature detection that will help us to solve problems. The tasks in this workshop are all around the theme of *bioinformatics* but could be applied in other domains.

Part 1: Plot an intensity profile of an image

As shown in the lectures we're going to make a plot to investigate our data. We are going to use a Python package called Matplotlib, but I will provide this code to you.

Download the image *barbilophozia_floerkei_cells_895150.jpg* from Blackboard for this. Start with this outline and complete the missing parts:

```
import cv2 as cv
from matplotlib import pyplot as plt

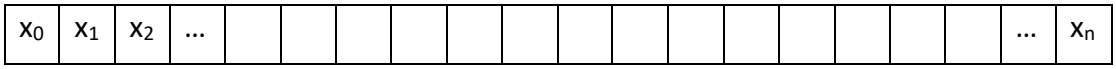
# ADD CODE HERE: read in the image in greyscale

# ADD CODE HERE: create a variable called row and set values

# plot
fig, ax = plt.subplots()
ax.plot(row)
ax.set(ylim=(0, 255))
plt.show()
```

The Matplotlib code is ready to take a row of intensity values in the variable called **row**. You need to:

1. Read in the image in greyscale;
2. Create **row** and set it to be an ROI that is the whole row wide but only one column tall so that you have a single strip of data like this, where each pixel has one value:



You can do this using the numpy array slicing e.g.

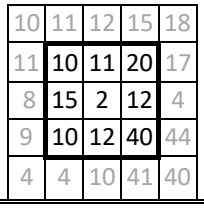
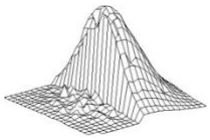
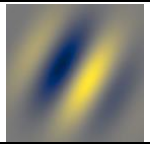
```
row = img[150, :]
```

Have a look through some different rows to see the different plots you can get.

They are currently very noisy, so we can try some of the things shown in the lectures to bring out the edges in the data.

Part 2: Applying Filters

Let's get convolutional and try applying filters to our image! We have a good set of preprepared filters available within OpenCV to use:

Name	Example (note, can be any size)	Description
Box	$\begin{bmatrix}1 & 1 & 1\\1 & 1 & 1\\1 & 1 & 1\end{bmatrix}$ A simple unnormalised box filter.	As you move the filter take the sum of all the pixels under it. This will both blur and brighten the image.
Normalised box	$\begin{bmatrix}1/9 & 1/9 & 1/9\\1/9 & 1/9 & 1/9\\1/9 & 1/9 & 1/9\end{bmatrix}$ We use $1/9$ because there are 9 pixels, if it were a 5x5 filter it would be $1/25$ etc.	Same as box but with added normalising fraction, so takes an average of the pixels under it. Will blur the image.
Median	Given the patch below, put all the values in the neighbourhood in order i.e. 2, 10, 10, 11, 12, 12, 15, 20, 40 Output the median value = 12. 	Outputs the median of the neighbourhood at each pixel. Good for removing speckles / salt and pepper noise.
Gaussian	3x3 approximation $\begin{bmatrix}0.075 & 0.124 & 0.075\\0.124 & 0.204 & 0.124\\0.075 & 0.124 & 0.075\end{bmatrix}$	Forms a weghted average of the neighbours, with a nice, smooth transition to a central peak, as seen in the lectures.
Laplacian	$\begin{bmatrix}0 & 1 & 0\\1 & -4 & 1\\0 & 1 & 0\end{bmatrix}$	A discrete approximation to the sum of 2nd derivatives in the x and y directions, as seen in the lectures.
Sobel	$\begin{bmatrix}1 & 0 & -1\\2 & 0 & -2\\1 & 0 & -1\end{bmatrix}$	A discrete approximation to the 1st derivative, as seen in the lectures.
Bilateral[1]		Essentially a combination of different filters; the webpage the image is from (link below) has a lot more info.
Scharr	$\begin{bmatrix}3 & 0 & -3\\10 & 0 & -10\\3 & 0 & -3\end{bmatrix}$	Similar to Sobel, more effective in some cases.
Gabor[2]		An oriented sine wave multiplied by a Gaussian.

[1] Image from: https://homepages.inf.ed.ac.uk/rbf/CVonline/LOCAL_COPIES/MANDUCHI1/Bilateral_Filtering.html

[2] Image from: https://en.wikipedia.org/wiki/Gabor_filter

Here are examples of some of the functions in code, with their arguments in the comments:

```
# image, kernel size (x,y)
result = cv.blur(img,(5,5))
```

```
# image, kernel size
result = cv.medianBlur(img,5)
```

```
# image, kernel size (x,y), sigma
result = cv.GaussianBlur(img,(3,3),1)
```

Apply each of these to your image from part 1, and do two things:

1. **Show the image on screen** with the filters applied so you can see the visual effect;
2. **Plot the intensity profile** of the filtered image so you can see the effect on the data.

Look through the OpenCV filter docs and have a go at using some of the other filters[3]:

https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html#gad78703e4c8fe703d479c1860d76429e6

Next let's see Sobel and Laplacian filters.
Sobel has multiple arguments we use to get the different orientations, the OpenCV implementation can do a lot more than we are going to use:

```
# image, output bit depth, dx, dy, kernel size
result = cv.Sobel(img, cv.CV_64F, 0, 1, 3)
```

The three new parameters are:

- **Output bit depth** – we are using 64-bit floating point values, cv.CV_64F, to allow the output to store both positive and negative values[4].
- **dx, dy** – is your choice of how many levels of differentiation on each axis; for this module we are only going to use what we saw in the lectures, set:
    - **dx = 1, dy = 0** for your derivative in the x-direction
    - **dx = 0, dy = 1** for your derivative in the y-direction

Try these out to get your two Sobel images.

Laplacian is a bit simpler, as it has fewer required arguments:

```
# image, output bit depth
result = cv.Laplacian(img, cv.CV_64F)
```

Try this and see how it immediately gets both directions – very neat!

[3] The OpenCV Filtering Tutorial is helpful too: https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html
[4] This tutorial will be helpful: https://docs.opencv.org/4.x/d5/d0f/tutorial_py_gradients.html

Part 3: Play with Canny

Now let's build on filtering to detect some edges! Canny is built into OpenCV and is super easy to use – it only requires three parameters to get started:

```
# input image
# threshold1: automatically remove anything below this value
# threshold2: only keep connected lines below this value

result = cv.Canny(img,threshold1,threshold2)
```

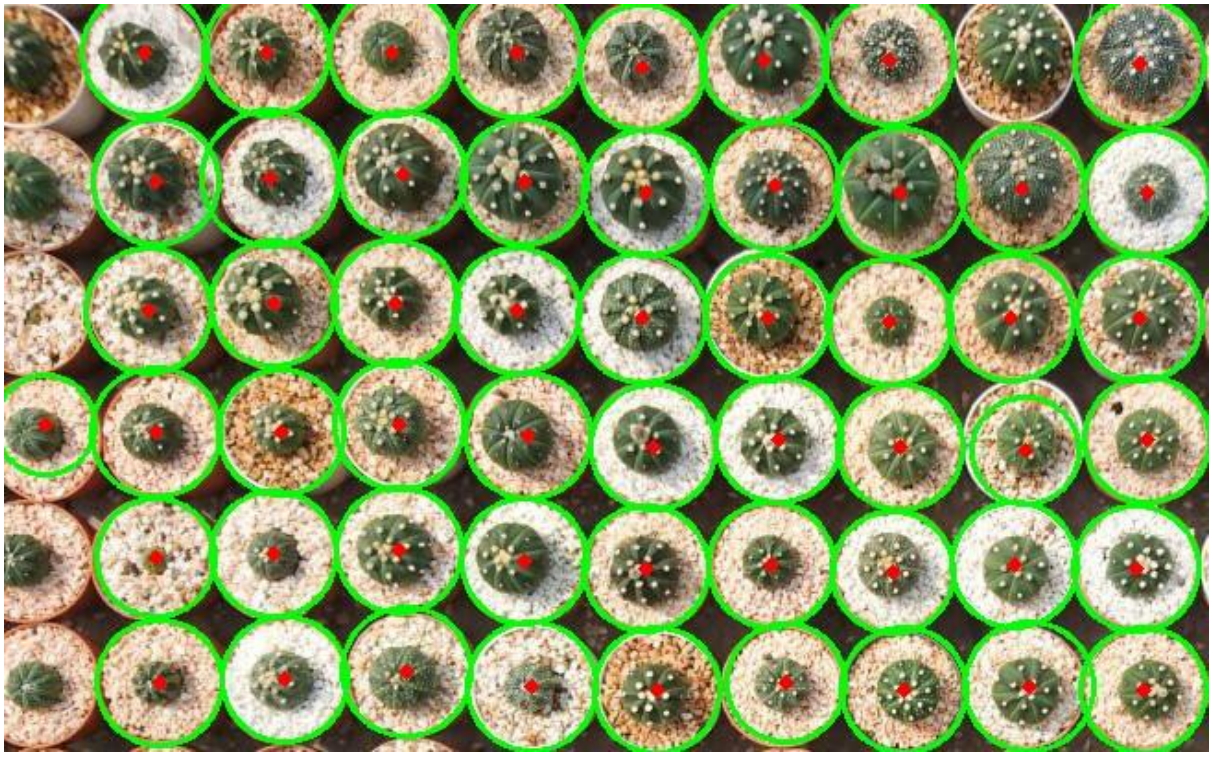
Thresholds 1 and 2 are part of the hysteresis processing – this is explained nicely in the Computerphile video linked on Blackboard, and in the OpenCV Canny Tutorial: https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html

Apply Canny to your image and play around with the threshold values. Remember I gave you some code to tweak the values using trackbars, you might like to use this here: https://github.com/lionspaws/CS34110-Computer-Vision/blob/main/canny-demo.py

Canny wasn't the only edge detection we use, let's see Hough in action next…

Part 4: Catch the Cacti

It's common to use feature detection to set some context for further work. Here we are going to try to find the pots in the image *cacti.jpg*. This will require implementing Hough Circles then tweaking the parameters to get the best result. This is the best I got, can you do better?



To implement this, take a look at the OpenCV tutorial on Hough Circles: https://docs.opencv.org/4.x/da/d53/tutorial_py_houghcircles.html

Start with this code; **beware** that you may need to sort out your indentation before Python will run it happily. Note two particular lines of code in the tutorial which we are going to experiment with:

```
img = cv.medianBlur(img,5)

circles = cv.HoughCircles(img,cv.HOUGH_GRADIENT,1,20, \
          param1=50,param2=30,minRadius=0,maxRadius=0)
```

The first line is applying a blur to the image (look back at the earlier part of the tutorial for more info); the second line is applying the Hough Circles algorithm to the image and comes with lots of parameters for us to tweak through arguments to the function. Let's break this function down a bit more to see what the arguments are:

```
circles = cv.HoughCircles(
     img,                 # your image
     cv.HOUGH_GRADIENT,   # detection method
     1,                   # accumulator param
     20,                  # minimum distance between circles
     param1=50,           # threshold for Canny
     param2=30,           # accumulator param
     minRadius=0,         # smallest circle size allowed
     maxRadius=0)         # largest circle size allowed (0=inf)
```

We only want to work with some of these here, but if you're interested and want to learn more, the full description of the function and its arguments is here: https://docs.opencv.org/4.x/dd/d1a/group__imgproc__feature.html#ga47849c3be0d0406ad3ca45db65a25d2d

Try changing the following parameters to see what you can do:

- Minimum distance between the circles: if too small you get lots of overlap, too much and you lose detections.
- minRadius: this is in pixels, and is the radius (i.e. half the width of the whole circle); any smaller circle detections will be filtered out.
- maxRadius: again, in pixels and is specifically the radius; a value of 0 means 'any size'.

Note that the first three arguments are required, but the others are optional.

Try using different blurring methods, and tweak the Hough Circle parameters to get your best match for the circular pots.

Beware that Hough Circles can take a lot of processing power if you have the wrong parameters, and it might take a few seconds to get a result.

Once you are happy with the circle detection, try counting the number of green pixels in each circle. Try to threshold the green areas by converting the image to the HSV colour

space (see previous worksheet!) and then use the `inRange` function to select the green pixels as a mask and the `countNonZero` function to count the number of green pixels i.e.

```
import numpy as np
green_lower = np.array([30, 10, 10])
green_upper = np.array([140, 200, 200])
thresh = cv2.inRange(imghsv, green_lower,green_upper)
count = cv2.countNonZero(thresh)
```

The code above will count ***all*** the green pixels in the image. To count the number in each cactus, you could add some code inside the loop over circles to create a circular mask image for one plant, combine it with the green thresholded mask using bitwise and, and then count the remaining green pixels. i.e.

```
    mask = np.zeros(thresh.shape, dtype=np.uint8)
    cv.circle(mask, (i[0],i[1]),i[2],255, -1)
    plant_pix = cv.bitwise_and(mask, thresh)
    pix = cv.countNonZero(plant_pix)
    cv.putText(img, str(pix), (i[0]-10,i[1]),
               cv.FONT_HERSHEY_SIMPLEX, 0.5,
               (0,255, 0),1,cv.LINE_AA)
    count += pix
    num = num+1.0
```

Part 5: Corners

Finally, let's start to find corners! Both Harris[5] and KLT[6] are implemented in OpenCV, and again are quite straightforward to use, but you need to do a little extra work to get them to show results as they are designed to process things internally as part of a chain.

Download the image *green_leaf_closeup_0.jpg* from Blackboard for this. Start with this outline[7] and complete the missing parts:

[5] OpenCV Harris Corners Tutorial: https://docs.opencv.org/4.x/dc/d0d/tutorial_py_features_harris.html
[6] OpenCV Shi-Tomasi (aka KLT) Tutorial: https://docs.opencv.org/4.x/d4/d8c/tutorial_py_shi_tomasi.html
[7] Based on OpenCV tutorial and https://medium.com/@deepanshut041/introduction-to-harris-cornerdetector-32a88850b3f6 (which has a more understandable approach to the thresholding post-processing)

```
import cv2 as cv
import numpy as np

# ADD CODE HERE: read in the image in colour, call it imgCol
# ADD CODE HERE: create a greyscale copy of the image as img

# we run Harris, first making sure the image is in the right
# format, then give the Harris parameters
imgHC = cv.cornerHarris(np.float32(img),2,3,0.04)

# dilate the result[8] (don't worry about this step)
imgHC = cv.dilate(imgHC, None)

# Threshold for an optimal value, calculated using
# the brightest value from the Harris Corners result
thresh = 0.1 * imgHC.max()

# Create copy of the colour image to draw corners on
result = imgCol

# Iterate through all the corners and draw them on the image
#(if they pass the threshold)
 for j in range(0,imgHC.shape[0]):
    for i in range(0, imgHC.shape[1]):
      if(imgHC[j,i] > thresh):
           # image, center pt, radius, color, thickness
           cv.circle( result, (i, j), 1, (0,255,0), 1)
# ADD CODE HERE: show the image in a Python window
```

The three parameters being set for Harris are:

```
result = cv.cornerHarris(image, blockSize, ksize, k)
```

- blockSize: size of the neighbourhood
- kSize: the size of the kernel for the Sobel operator that it uses internally
- k: part of the optimization process; don't worry too much about it here, but footnote 7 has a bit more detail if you're interested.

The only parameter that we're going to tweak is the kSize. It must be an odd number and can't be more than 31 – try changing it to see what results you can get. Now try KLT:

[8] Note: if you want to learn more about dilation, this is a nice explanation: https://uk.mathworks.com/help/images/morphological-dilation-and-erosion.html

```
import cv2 as cv
import numpy as np

# ADD CODE HERE: read in the image in colour, call it imgCol
# ADD CODE HERE: create a greyscale copy of the image as img
# KLT
corners = cv.goodFeaturesToTrack(img,25,0.01,10)
corners = np.int0(corners)

result = imgCol
for i in corners:
    x,y = i.ravel()
    cv.circle(imgCol,(x,y),3,255,-1)

# ADD CODE HERE: show the image in a Python window
```

This is a lot simpler, and in fact we can use Harris via this function too… The basic parameters (required arguments) used in the above are as follows:

```
# image, maxCorners, qualityLevel, minDistance
corners = cv.goodFeaturesToTrack(img,25,0.01,10)
```

So we are limiting the algorithm to the best 25 corners, and making sure they are spaced out by 10 pixels. The quality measure is a bit more complex but you can read about that in the OpneCV docs:
https://docs.opencv.org/4.x/dd/d1a/group__imgproc__feature.html#ga1d6bb77486c8f92d79c8793ad995d541

There are some optional arguments to try, including switching whether it uses Harris or an alternative as the base detector (remember, KLT mostly replaces the optimisation function originally used in Harris):

```
corners = cv.goodFeaturesToTrack(..., useHarrisDetector=True)
```

Try using Harris or the alternative (which looks for minimum eigenvalues in the gradients – don't worry about this, but you can read up if you're interested); see what different results you can get.

Link to the Assignment

In the assignment you are asked to detect yellow cars. Consider the following questions relating to the assignment, and write down your answers, you will need them for the assignment report.

CS34020 and CSM4220 Students

What Python / opencv functionality from this workshop could be used to:

- segment the yellow pixels in an image or part of an image?

- Calculate the percentage of yellow pixels in a (sub)image?
- Detect some features on a car that could help with tracking the car?

CSM4220 Students only

In addition to the above, find a published academic computer science paper on **vehicle colour** recognition / classification, and briefly summarise it and cite the paper.